

National Institute of Technology Karnataka Surathkal
Department of Information Technology



IT 252 DATABASE SYSTEMS

The Relational Data Model and SQL

Dr. Jayashree T R

Course Outline

Course Plan: Theory:

Part A: Parallel Computer Architectures

Week 1: Introduction to to the course, highlighting the data and databases, and related basic concepts. Advantages and need for a database system.

Week 2: Attributes, tuples, relational schema, conceptual model, introduction to SQL-DML, DDL, creating relations/ tables, access and manipulate data.

Week 3,4: Data model, relational model, and SQL, relational data model and relational database constraints, SQL data definition and data types, specifying constraints, basic retrieval queries, complex quires, triggers, views, schema modification.

Week 5-6: ER model development, entity types/sets, attribute, relationship types/sets, simple employee database conceptual design using ER concepts, ER to relational mapping algorithm, EER model.

Course Outline

- Week 7,8 : Functional dependency definition, need of normalization, anomalies and redundant information in tuples, normalization steps (1NF, 2NF, 3NF), BCNF, introduction to the higher normal forms.
- Week 9: Procedural query languages: relational algebra and SQL-SELECT, PROJECT, RENAME, binary and JOIN operations with examples.
- Non-Procedural query languages: relational calculus, and SQL-tuple relational calculus, existential and universal quantifier with examples.
- Week 10,11 : Transaction management, schedule and serializability, concurrency control, 2-phase lock, recovery mechanism: undo/redo values.
- Week 12,13: Disk storage- basic file structures, ordered/unordered. binary search, hashing, indexing, importance of indexes, primary and secondary indexing methods, clustered, B tree, B++ trees.
- Week 14: Current trends in database system, introduction to data warehousing, data mining.
- Practical: MySQL commands
- Project: Team of 2 or 3 members

Relational Model Concepts

- The Relational model of data is based on the concept of a **Relation**.
- This Model uses the concept of a **Mathematical relation** , which looks like a **table of values** .
- A **Relation** is a mathematical concept based on the idea of sets.
- The model was first proposed by Dr. E.F. Codd of IBM Research in 1970 .
- The 1st commercial implementation of the relational model became available in 1980s . (IBM DB2)

Relational Model Concepts

- The relational model represents the database as a collection of relations .
- Each relation resembles a Table of Values and typically contains a set of rows.
- **A relation** is unordered set that contain the relationship of **attributes** that represent **entities**.
- A **tuple** is a set of attribute values (also known as its domain) in the relation. i.e. In the formal model, rows are called **tuples**.
 - Values are (normally) atomic/scalar.---→ **each value in the domain is indivisible**
 - The special value NULL is a member of every domain.

Relational Model Concepts

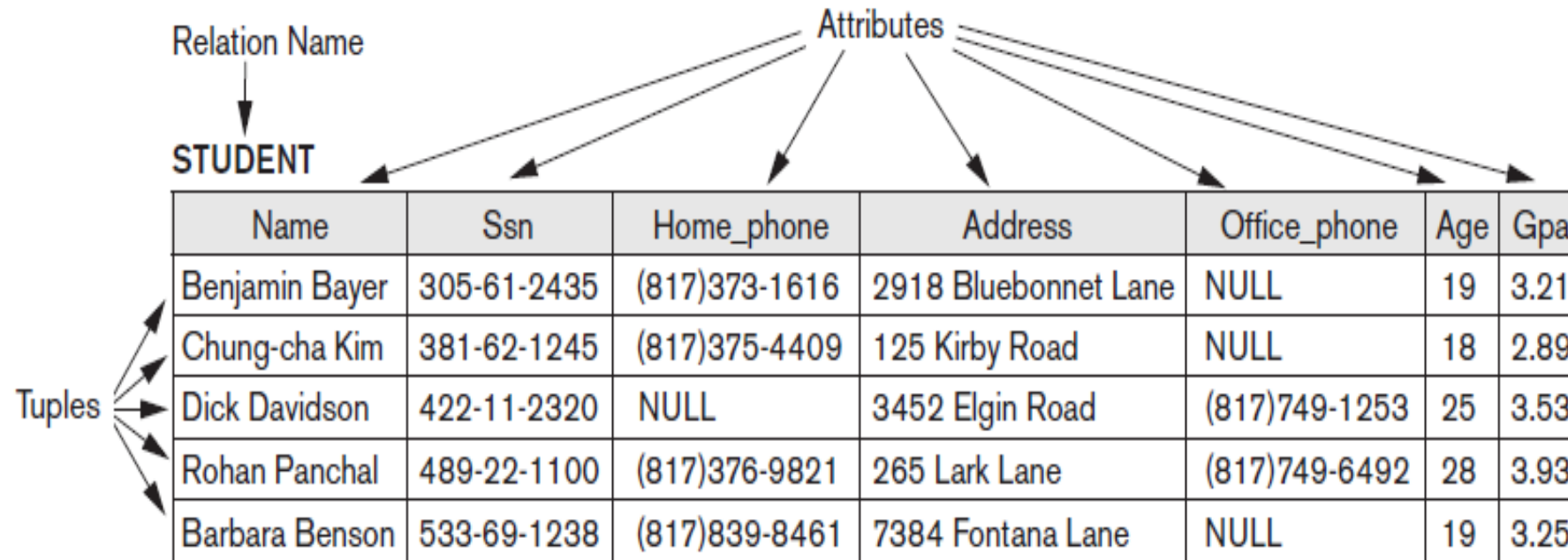


Figure 5.1

The attributes and tuples of a relation **STUDENT**.

Relational Model Concepts

- Each column has a column header that specify how to interpret the Data Values in each row. In the formal model, the column header is called an **attribute** name (or just attribute).
- **Domain (D)** - A **domain** in the relational model refers to the set of all possible values (i.e. a set of Atomic values) that a column or attribute can have.
- We generally give a name to the domain , because it helps in interpreting its values. A data type or format is also specified for each domain.

Artist(name, year, country)

name	year	country
Wu Tang Clan	1992	USA
Notorious BIG	1992	USA
Ice Cube	1989	USA

n-ary Relation

=

Table with n columns

Relational Model Concepts

e.g. : “phone_numbers_of India ” are the set of 10 digit mobile numbers valid in the India. The phone_numbers_ofIndia may have a format: (ddd) ddd-dddd where each d is a decimal digit.

Relational Model Concepts: The Schema (or description) of a Relation:

- A relation schema is used to describe a relation.
- Denoted by $R(A_1, A_2, \dots, A_n)$
- R is the **name** of the relation
- D is called the domain of A_i and is denoted by $\text{dom}(A_i)$.
- The **attributes** of the relation are $A_1, A_2, \dots, A_n$
- A relation schema $R(A_1, A_2, \dots, A_n)$, is made up of a relation named R and a list of attributes $A_1, A_2, \dots, A_n$.
- Each attribute A_i is the name of a role played by some domain D in the Relation Schema R .

Relational Model Concepts: The Schema (or description) of a Relation:

- **The degree of a relation** is the number of attributes n of its relation schema .
- Example:
CUSTOMER (Cust-id, Cust-name, Address, Phone#)
 - CUSTOMER is the relation name
 - Defined over the four attributes: Cust-id, Cust-name, Address, Phone#
- Each attribute has a **domain** or a set of valid values.
 - For example, the domain of Cust-id is 6 digit numbers.

RELATIONAL DATABASE SCHEMA

- **Relational Database Schema:**
 - A set S of relation schemas that belong to the same database.
 - S is the name of the whole **database schema**
 - $S = \{R_1, R_2, \dots, R_n\}$
 - $R_1, R_2, \dots, R_n$ are the names of the individual **relation schemas** within the database S
- Following slide shows a COMPANY database schema with 6 relation schemas

Relational Model Concepts: Relational Database Schema

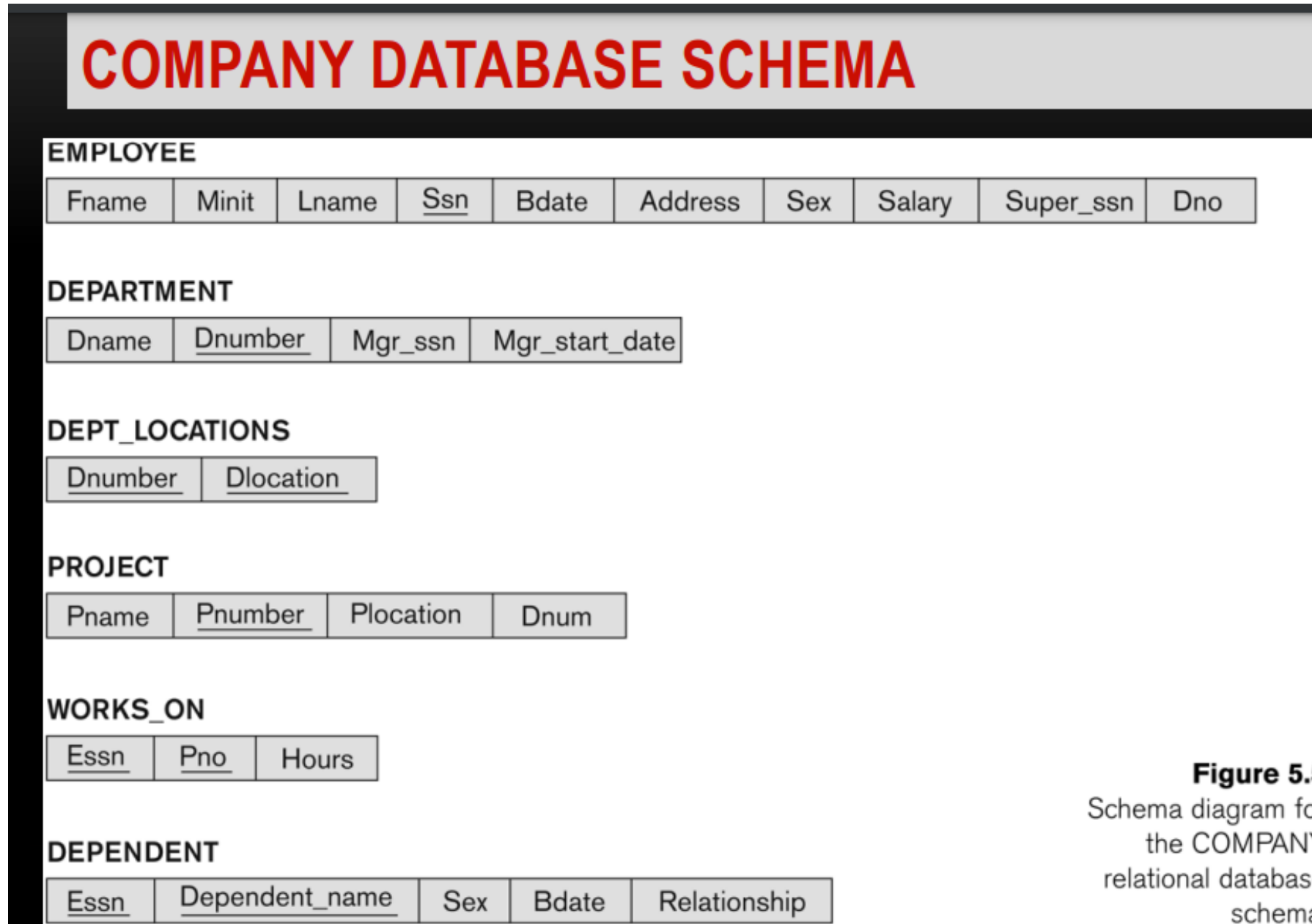


Figure 5.5
Schema diagram for
the COMPANY
relational database
schema.

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

- ER diagram is converted into the tables in relational model.
- This is because relational models can be easily implemented by RDBMS like MySQL , Oracle, PostgreSQL etc.

ER-to-Relational Mapping Algorithm

Step 1: Mapping of Regular Entity Types

Step 2: Mapping of Weak Entity Types

Step 3: Mapping of Binary 1:1 Relation Types

Step 4: Mapping of Binary 1:N Relationship Types.

Step 5: Mapping of Binary M:N Relationship Types.

Step 6: Mapping of Multi valued attributes.

Step 7: Mapping of N-ary Relationship Types.

ER DIAGRAM – Relationship Types are: WORKS_FOR, MANAGES, WORKS_ON, CONTROLS, SUPERVISION, DEPENDENTS_OF

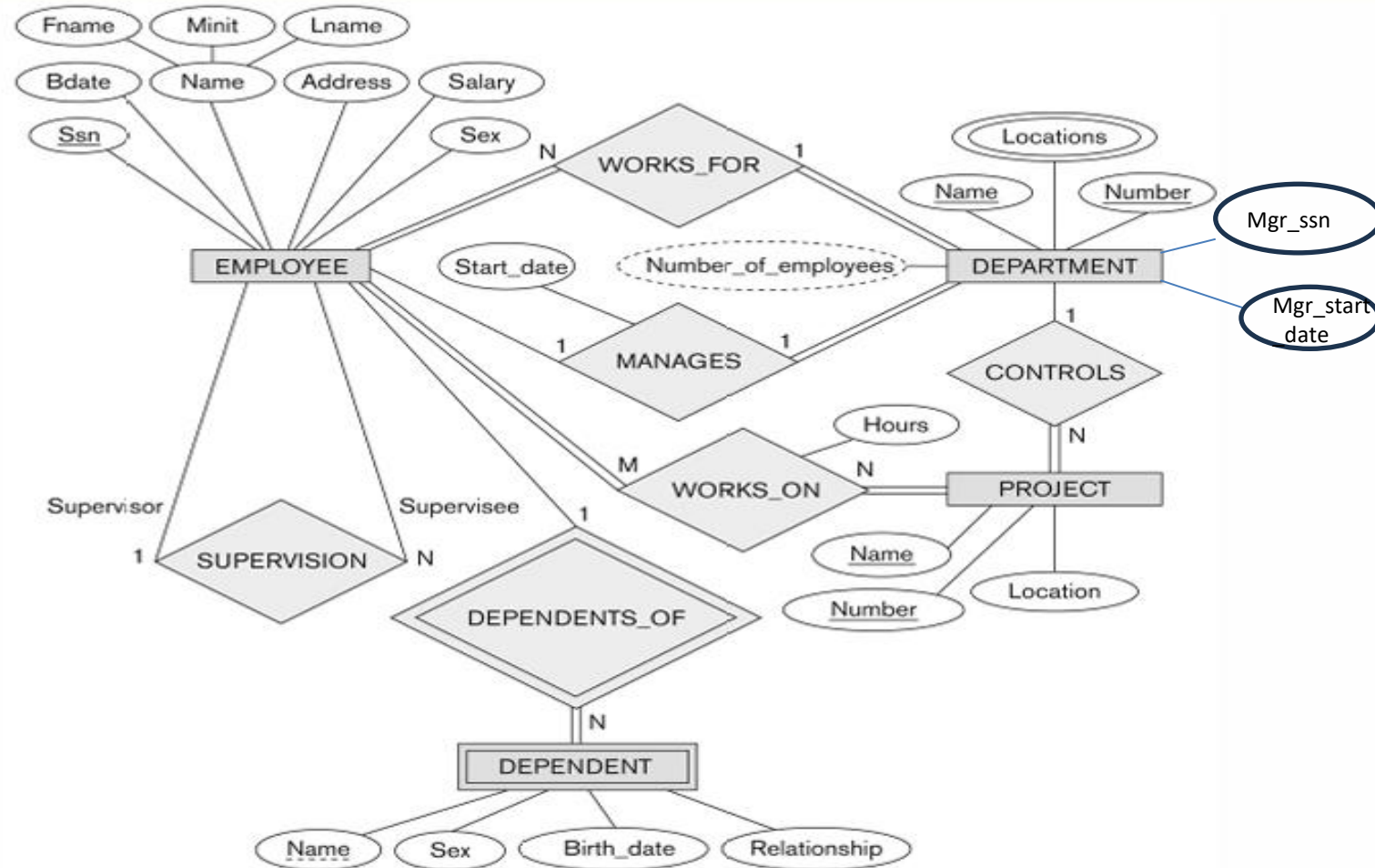


Figure 3.2

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

Step 1: Mapping of Regular Entity Types

- For each **regular (strong)** entity type E in the ER schema, create a relation R that includes all the simple attributes of E.
- Choose one of the key attributes of E as the primary key for R. If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R.

Example: Relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram. SSN, DNUMBER, and PNUMBER are primary keys for relations EMPLOYEE, DEPARTMENT, and PROJECT

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation
-------	----------------	-----------

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

- **Step 2: Mapping of Weak Entity Types**

- I. For each weak entity type W in the ER schema with owner entity type E, create a relation R and include all simple attributes (or simple components of composite attributes) of W as attributes of R.
- II. In addition, include as foreign key attributes of R the primary key attribute(s) of the relation(s) that correspond to the owner entity type(s).
- III. The primary key of R is the combination of the primary key(s) of the owner(s) and the partial key of the weak entity type W, if any.

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Essn
references
Ssn
EMPLOYEE

Ex: for weak entity type DEPENDENT, the relation is DEPENDENTS_OF.
Include SSN of EMPLOYEE relation as foreign key attribute of DEPENDENT (renamed to ESSN).
primary key of DEPENDENT relation is combination {ESSN, DEPENDENT_NAME} because
DEPENDENT_NAME is partial key of DEPENDENT.

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

ER-to-Relational Mapping Algorithm

- **Step 3: Mapping of Binary 1:1 Relation Types**
 - For each binary 1:1 relationship type R in ER schema, identify relations S and T, entity types participating in R.
- One approach (**Foreign Key approach**):
 - Choose one of relations -say S- and include as a foreign key in S the primary key of T. It is better to choose an entity type with total participation in R in the role of S.
- Example: 1-to-1 relation MANAGES is mapped by choosing the participating entity type DEPARTMENT to serve in the role of S, because its participation in the MANAGES relationship type is total.

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
-------	---------	---------	----------------

Mgr_ssn
references SSN
of EMPLOYEE

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

ER-to-Relational Mapping Algorithm

Step 4: Mapping of Binary 1:N Relationship Types.

- For each regular binary 1:N relationship type R (*from T to S*), identify the relation S that represent the participating entity type at the N-side of the relationship type.
- Include as foreign key in S the primary key of the relation T that represents the other entity type participating in R.
- Include any simple attributes of the 1:N relation type as attributes of S.

Example: 1-to-N relationship types WORKS_FOR, CONTROLS, and SUPERVISION in the figure.

- For WORKS_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.



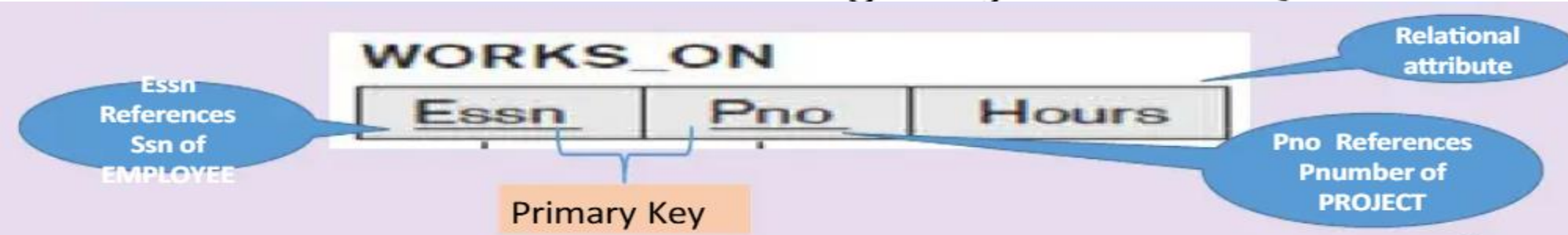
ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

Step 5: Mapping of Binary M:N Relationship Types

- I. For each regular binary M:N relationship type R, **create a new relation** S to represent R.
- II. Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; **their combination will form the primary key** of S.
- III. Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S.

Example: The M:N relationship type WORKS_ON from the ER diagram is mapped by creating a relation WORKS_ON in the relational database schema.

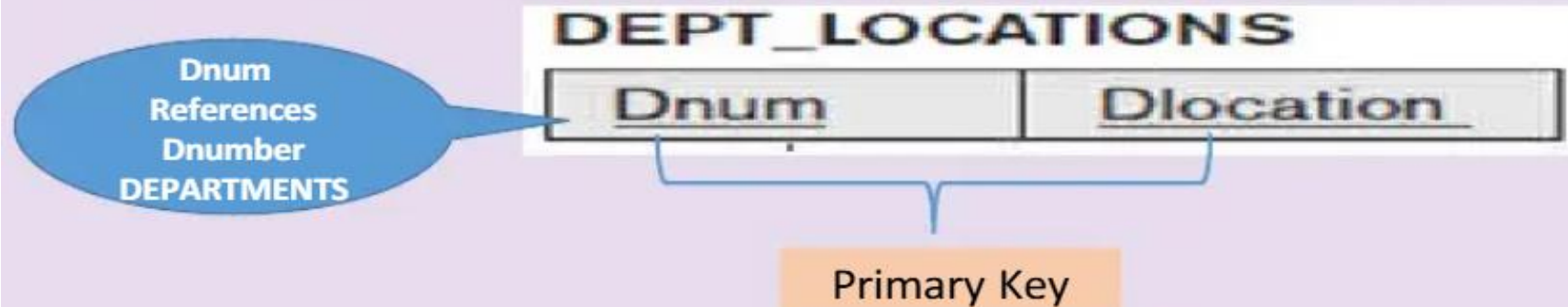
The primary keys of the PROJECT and EMPLOYEE relations are included as foreign keys in WORKS_ON and renamed PNO and ESSN, respectively. Attribute HOURS in WORKS_ON represents the HOURS attribute of relation type. The primary key of the WORKS_ON relation is combination of the foreign key attributes {ESSN, PNO}.



ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

Step 6: Mapping of Multivalued attributes

- I. For each **multivalued attribute** A, create a new relation R. This relation R will include an attribute corresponding to A, plus the primary key attribute K-as a foreign key in R-of the relation that represents the entity type of relationship type that has A as an attribute.
- II. The primary key of R is the combination of A and K. If the multivalued attribute is composite, we include its simple components.



Example: The relation DEPT_LOCATIONS is created.

Attribute DLOCATION represents multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation. The primary key of R is the combination of {DNUMBER, DLOCATION}.

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

Step 7: Mapping of N-ary Relationship Types

- I. For each n-ary relationship type R, where $n > 2$, **create a new relationship S** to represent R.
- II. Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- III. Also include any simple attributes of the n-ary relationship type (or simple components of composite attributes) as attributes of S.

Example: The relationship type SUPPY in the ER on the next slide.

This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME} in case the cardinalities are N:N:N

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

ER-to-Relational Mapping Algorithm-step 7 example



SUPPLIER

<u>SNAME</u>	...
--------------	-----

PROJECT

<u>PROJNAME</u>	...
-----------------	-----

PART

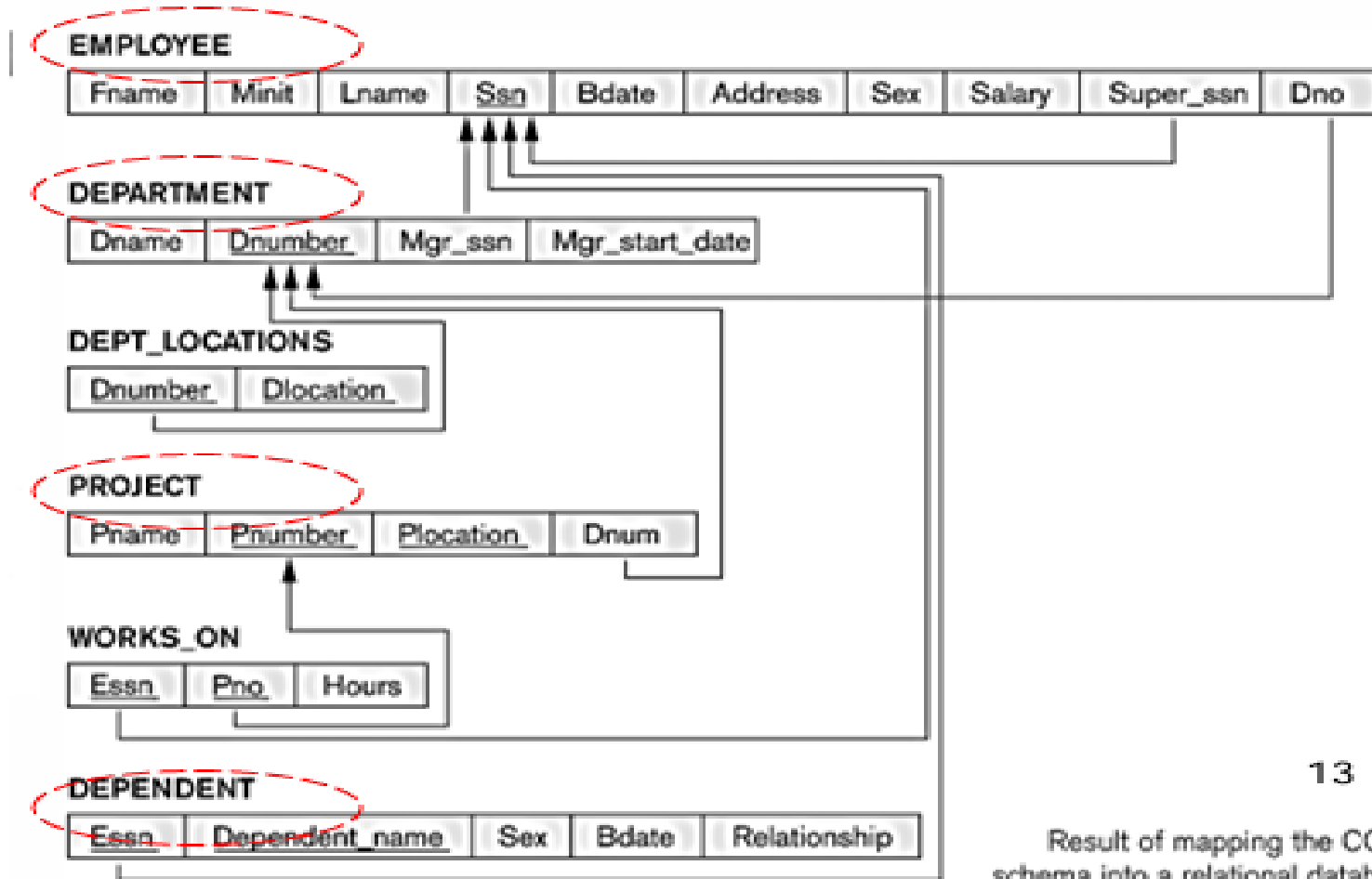
<u>PARTNO</u>	...
---------------	-----

SUPPLY

<u>SNAME</u>	<u>PROJNAME</u>	<u>PARTNO</u>	QUANTITY
--------------	-----------------	---------------	----------

Note that PK depends on the cardinalities of SUPPLY

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)



13

Figure 7.2
Result of mapping the COMPANY ER
schema into a relational database schema.

ER to Relational Mapping Algorithm (Reduction of ER diagram to Table)

Summary of Mapping constructs and constraints

Table 7.1 Correspondence between ER and Relational Models

ER Model

Entity type

1:1 or 1:N relationship type

M:N relationship type

n -ary relationship type

Simple attribute

Composite attribute

Multivalued attribute

Value set

Key attribute

Relational Model

“Entity” relation

Foreign key (or “relationship” relation)

“Relationship” relation and two foreign keys

“Relationship” relation and n foreign keys

Attribute

Set of simple component attributes

Relation and foreign key

Domain

Primary (or secondary) key

Relational Model Concepts: Contd..

Values and NULLs in the Tuples

- NULL values are used to represent the values of attributes that may be unknown or may not apply to a tuple.
- **value unknown, value exists but is not available, or attribute does not apply (value undefined)**
- During database design, it is best to avoid NULL values as much as possible.

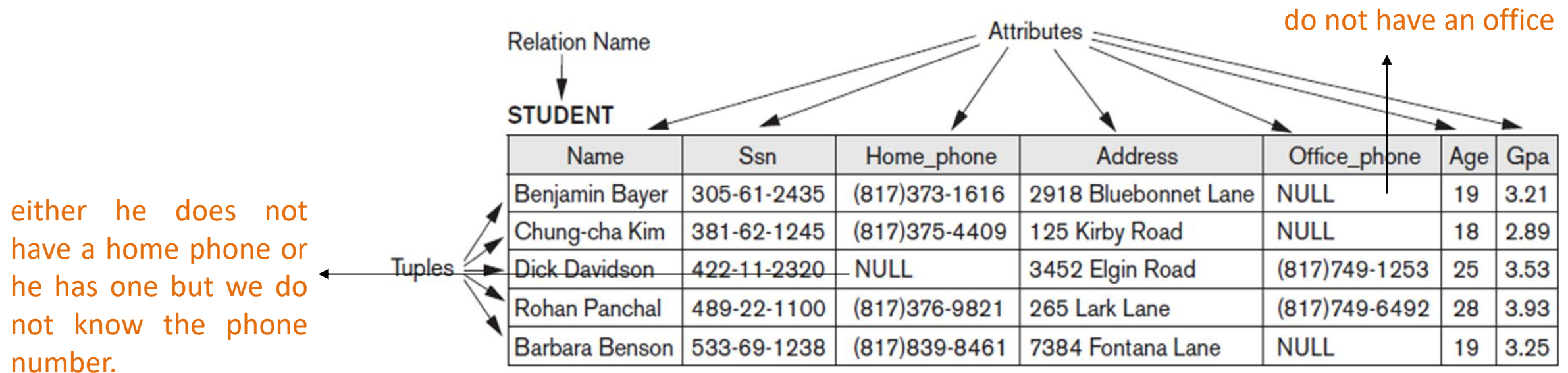


Figure 5.1

The attributes and tuples of a relation STUDENT.

Entity Integrity, Referential Integrity, and Foreign Keys

The **entity integrity constraint** states that *no primary key* value can be NULL. This is because the primary key value is used to identify individual tuples in a relation. Having NULL values for the primary key implies that we cannot identify some tuples.

Key constraints and entity integrity constraints are specified on individual relations.

Referential integrity constraints work on the concept of Foreign Keys.

The **referential integrity constraint** is specified between two relations and is used to maintain the consistency among tuples in the two relations. i.e., the referential integrity constraint states that a tuple in one relation that refers to another relation must refer to an existing tuple in that relation.

Entity Integrity, Referential Integrity, and Foreign Keys

For example, the attribute *Dno* of **EMPLOYEE** gives the department number for which each employee works; hence, its value in every **EMPLOYEE** tuple must match the *Dnumber* value of some tuple in the **DEPARTMENT** relation.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Entity Integrity, Referential Integrity, and Foreign Keys

Referential integrity : a Foreign Key perspective:

- A foreign key specifies that an attribute from one relation has to map to a tuple in another relation. i.e. *A foreign key is a key attribute of a relation that can be referred in another relation.*
- The conditions for a foreign key, specify a **referential integrity constraint** between the two relation schemas R1 and R2. A set of attributes FK in relation schema R1 is a foreign key of R1 that references relation R2 if it satisfies the following rules:
 - The attributes in FK have the same domain(s) as the primary key attributes PK of R2; the attributes FK are said to reference or refer to the relation R2.
 - A value of FK in a tuple t1 of the current state r1(R1) either occurs as a value of PK for some tuple t2 in the current state r2(R2) or is NULL. In the former case, we have $t1[FK] = t2[PK]$, and we say that the tuple t1 references or refers to the tuple t2.

Entity Integrity, Referential Integrity, and Foreign Keys

In this definition, **R1** is called the **referencing relation** and **R2** is the **referenced relation**. If these two conditions hold, a **referential integrity constraint from R1 to R2** is said to hold. In a database of many relations, there are usually many referential integrity constraints.

e.g.: In the EMPLOYEE relation, the attribute Dno refers to the department for which an employee works; hence, we designate Dno to be a foreign key of EMPLOYEE referencing the DEPARTMENT relation. This means that a value of Dno in any tuple t1 of the EMPLOYEE relation must match a value of the primary key of DEPARTMENT—the Dnumber attribute—in some tuple t2 of the DEPARTMENT relation, or the value of Dno can be NULL if the employee does not belong to a department.

Note: a foreign key can refer to its own relation.

e.g.: Super_ssn is a foreign key that references the EMPLOYEE relation itself. The tuple for employee 'John Smith' references the tuple for employee 'Franklin Wong,' indicating that 'Franklin Wong' is the supervisor of 'John Smith'.

Entity Integrity, Referential Integrity, and Foreign Keys

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Diagrammatic Representation of Referential integrity constraints displayed on the COMPANY relational database schema.

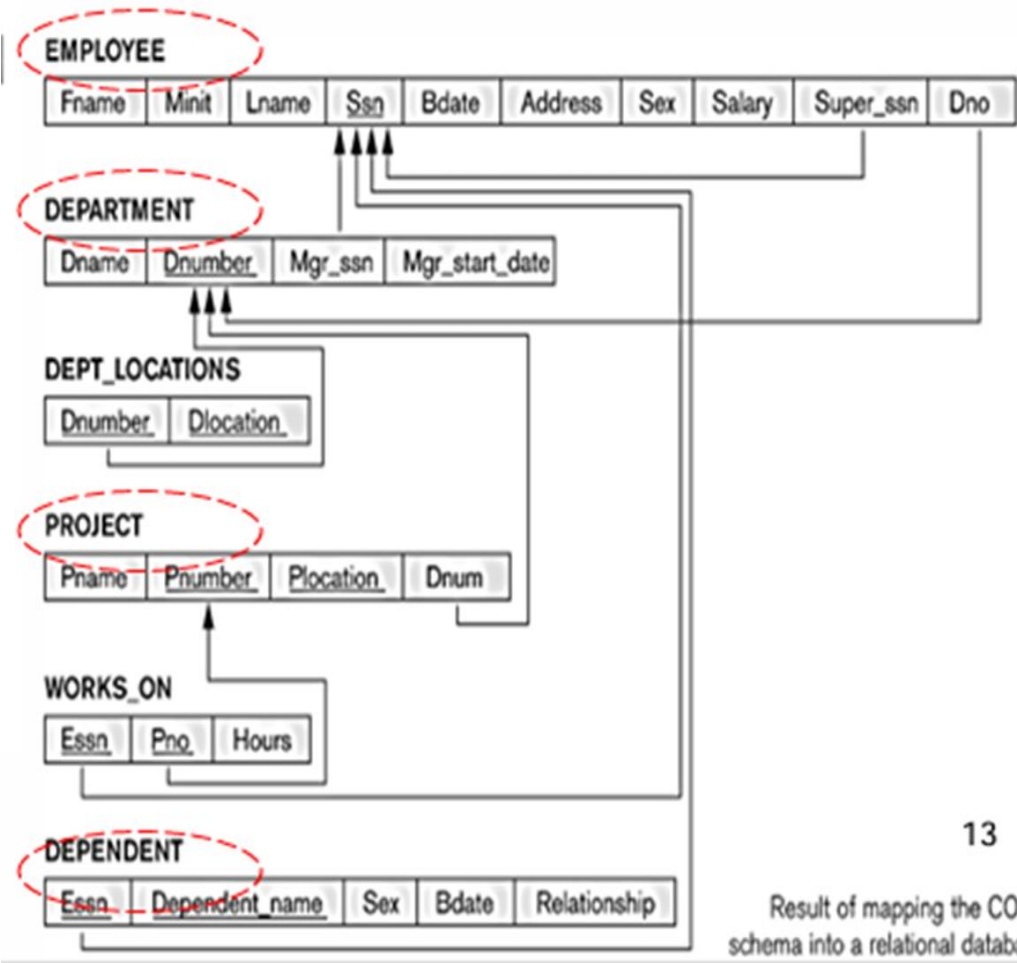


Figure 7.2
Result of mapping the COMPANY ER
schema into a relational database schema.

Basic SQL

- SQL language
 - Considered one of the major reasons for the commercial success of relational databases
- SQL
 - The origin of SQL is relational predicate calculus called tuple calculus which was proposed initially as the language SQUARE.
 - SQL Actually comes from the word “SEQUEL” which was the original term used in the paper: “SEQUEL TO SQUARE” by Chamberlin and Boyce. IBM could not copyright that term, so they abbreviated to SQL and copyrighted the term SQL.
 - Now popularly known as “Structured Query language”.
 - SQL is an informal or practical rendering of the relational data model with syntax

SQL Data Definition, Data Types, Standards

- Terminology:
 - **Table**, **row**, and **column** used for relational model terms relation, tuple, and attribute
- CREATE statement
 - Main SQL command for data definition
- The language has features for : Data definition, Data Manipulation, Transaction control (Transact-SQL, Ch. 20), Indexing (Ch.17), Security specification (Grant and Revoke- see Ch.30), Active databases (Ch.26), Multi-media (Ch.26), Distributed databases (Ch.23) etc.

SQL Standards

- SQL has gone through many standards: starting with SQL-86 or SQL 1.A. SQL-92 is referred to as SQL-2.
- Later standards (from SQL-1999) are divided into **core** specification and specialized **extensions**. The extensions are implemented for different applications – such as data mining, data warehousing, multimedia etc.
- SQL-2006 added XML features (Ch. 13); In 2008 they added Object-oriented features (Ch. 12).
- SQL-3 is the current standard which started with SQL-1999. It is not fully implemented in any RDBMS.

The current SQL standard in use is **SQL:2016** (also referred to as ISO/IEC 9075:2016).

- JSON Support
- Row Pattern Recognition
- Polymorphic Table Functions (PTFs)

Schema and Catalog Concepts in SQL

- **SQL schema**
 - Identified by a **schema name**
 - Includes an **authorization identifier** and **descriptors** for each element
- **Schema elements include**
 - Tables, constraints, views, domains, and other constructs
- Each statement in SQL ends with a **semicolon**

Schema and Catalog Concepts in SQL (cont'd.)

- **CREATE SCHEMA statement**

- `CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith' ;`

- **Catalog**

- Named collection of schemas in an SQL environment

- **SQL also has the concept of a cluster of catalogs.**

Note: Schema refer to the layout or structure of a database, which includes the tables, their columns, relationships, and constraints (in database systems such as PostgreSQL).

The CREATE TABLE Command in SQL

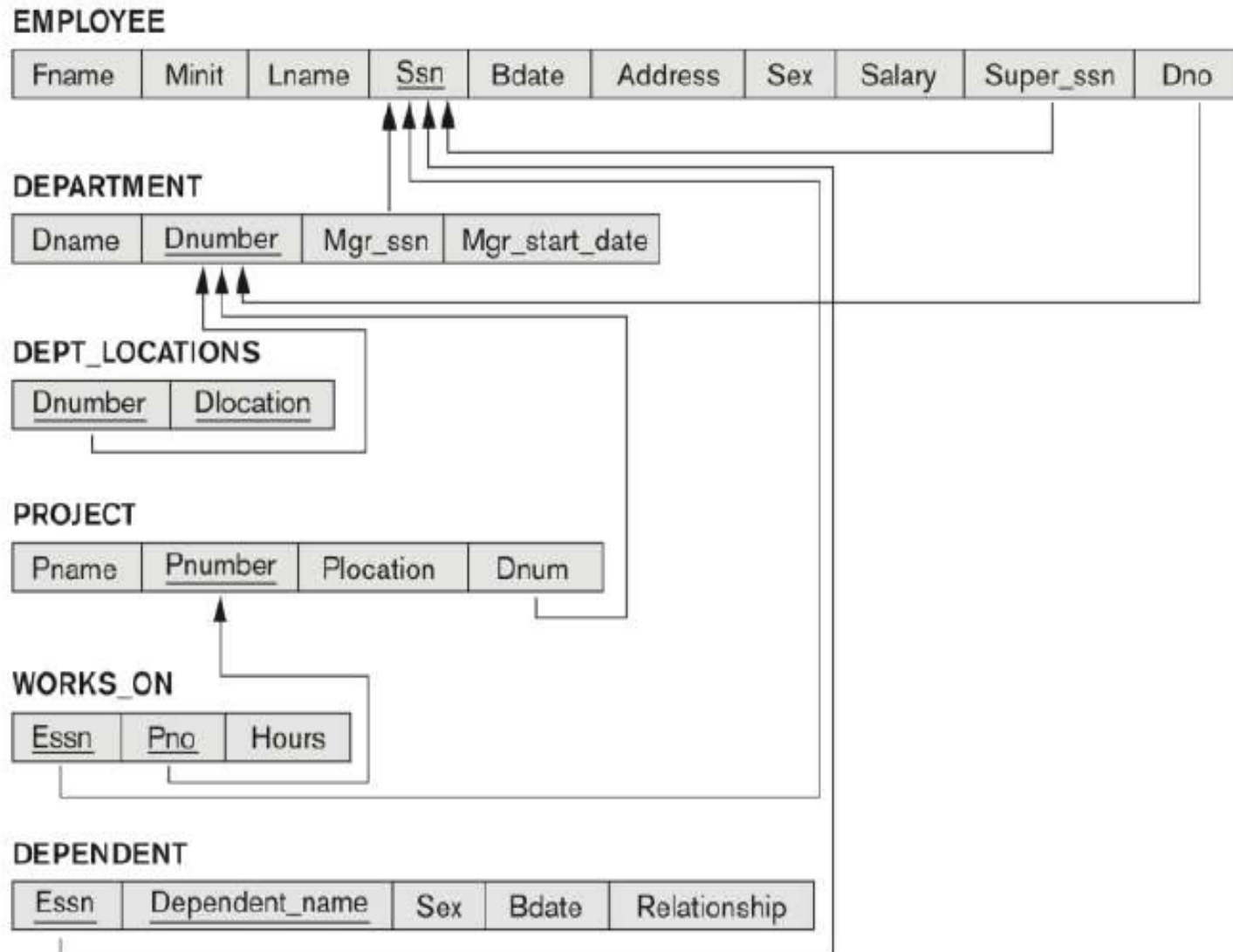
- Specifying a new relation
 - Provide name of table
 - Specify attributes, their types and initial constraints
- Can optionally specify schema:
 - `CREATE TABLE COMPANY.EMPLOYEE ...`
or
 - `CREATE TABLE EMPLOYEE ...`

The CREATE TABLE Command in SQL (cont'd.)

- **Base tables (base relations)**
 - Relation and its tuples are actually created and stored as a file by the DBMS
- **Virtual relations (views)**
 - Created through the `CREATE VIEW` statement. Do not correspond to any physical file.

CREATE VIEW – WILL BE DISCUSSED IN NEXT CHAPTER

COMPANY relational database schema (Fig. 5.7)



One possible database state for the COMPANY relational database schema (Fig. 5.6)

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

One possible database state for the COMPANY relational database schema – continued (Fig. 5.6)

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

<u>Pname</u>	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 5.7 (Fig. 6.1)

```
CREATE TABLE EMPLOYEE
( Fname          VARCHAR(15)          NOT NULL,
  Minit          CHAR,
  Lname          VARCHAR(15)          NOT NULL,
  Ssn            CHAR(9)              NOT NULL,
  Bdate          DATE,
  Address        VARCHAR(30),
  Sex            CHAR,
  Salary         DECIMAL(10,2),
  Super_ssn      CHAR(9),
  Dno            INT                  NOT NULL,
  PRIMARY KEY (Ssn),
CREATE TABLE DEPARTMENT
( Dname          VARCHAR(15)          NOT NULL,
  Dnumber        INT                  NOT NULL,
  Mgr_ssn        CHAR(9)              NOT NULL,
  Mgr_start_date DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
( Dnumber        INT                  NOT NULL,
  Dlocation      VARCHAR(15)          NOT NULL,
  PRIMARY KEY (Dnumber, Dlocation),
  FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

continued on ne

SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 5.7 (Fig. 6.1)-continued

```
CREATE TABLE PROJECT
  ( Pname                VARCHAR(15)          NOT NULL,
    Pnumber              INT                  NOT NULL,
    Plocation            VARCHAR(15),
    Dnum                 INT                  NOT NULL,
    PRIMARY KEY (Pnumber),
    UNIQUE (Pname),
    FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );

CREATE TABLE WORKS_ON
  ( Essn                 CHAR(9)              NOT NULL,
    Pno                  INT                  NOT NULL,
    Hours                DECIMAL(3,1)         NOT NULL,
    PRIMARY KEY (Essn, Pno),
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
    FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );

CREATE TABLE DEPENDENT
  ( Essn                 CHAR(9)              NOT NULL,
    Dependent_name        VARCHAR(15)         NOT NULL,
    Sex                   CHAR,
    Bdate                 DATE,
    Relationship            VARCHAR(8),
    PRIMARY KEY (Essn, Dependent_name),
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```


Attribute Data Types and Domains in SQL

- **Basic data types**

- **Numeric data types**

- Integer numbers: INTEGER, INT, and SMALLINT
 - Floating-point (real) numbers: FLOAT or REAL, and DOUBLE PRECISION

- **Character-string data types**

- Fixed length: CHAR(*n*), CHARACTER(*n*)
 - Varying length: VARCHAR(*n*), CHAR VARYING(*n*), CHARACTER VARYING(*n*)

Attribute Data Types and Domains in SQL (cont'd.)

- **Bit-string data types**
 - Fixed length: `BIT (n)`
 - Varying length: `BIT VARYING (n)`
 - **Boolean data type**
 - Values of `TRUE` or `FALSE` or `NULL`
 - **DATE data type**
 - Ten positions
 - Components are `YEAR`, `MONTH`, and `DAY` in the form `YYYY-MM-DD`
 - Multiple mapping functions available in RDBMSs to change date formats
-

Attribute Data Types and Domains in SQL (cont'd.)

- Additional data types

- **Timestamp** data type

- Includes the `DATE` and `TIME` fields

- Plus a minimum of six positions for decimal fractions of seconds
 - Optional `WITH TIME ZONE` qualifier

- **INTERVAL** data type

- Specifies a relative value that can be used to increment or decrement an absolute value of a date, time, or timestamp

- **DATE, TIME, Timestamp, INTERVAL** data types can be **cast** or converted to string formats for comparison.

Specifying Constraints in SQL

Basic constraints:

- Relational Model has 3 basic constraint types that are supported in SQL:
 - **Key** constraint: A primary key value cannot be duplicated
 - **Entity Integrity** Constraint: A primary key value cannot be null
 - **Referential integrity** constraints : The “foreign key” must have a value that is already present as a primary key, or may be null.

Specifying Attribute Constraints

Other Restrictions on attribute domains:

- Default value of an attribute

- **DEFAULT** <value>

- **NULL** is not permitted for a particular attribute (NOT NULL)

- **CHECK** clause

- Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);

In MySQL, the DEFAULT constraint is used to assign a default value to a column when no value is specified during an INSERT operation.

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    hire_date DATE DEFAULT (CURRENT_DATE),  
    status VARCHAR(20) DEFAULT 'active'  
);
```

```
INSERT INTO employees (employee_id, name)  
VALUES (1, 'John Doe'); -- hire_date will default to the  
current date, status will default to 'active'
```

In MySQL, the **CHECK constraint** is used to limit the values that can be inserted into a column, based on a specified condition or expression.

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    age INT,  
    salary DECIMAL(10, 2),  
    CHECK (age >= 18),           -- Ensures age is 18 or older  
    CHECK (salary >= 0)         -- Ensures salary is non-negative  
);
```

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    price DECIMAL(10, 2),  
    quantity INT,  
    CHECK (price >= 0 AND quantity >= 0)  
    -- Price and quantity must both be non-  
    negative)
```

```
INSERT INTO employees (employee_id, name, age, salary) VALUES (2, 'Jane Smith', 17, 3000.00);  
-- Error! Age must be >= 18
```


Specifying Key and Referential Integrity Constraints

■ PRIMARY KEY

- Specifies one or more attributes that make up the primary key of a relation
- `Dnumber INT PRIMARY KEY;`

■ UNIQUE

- Specifies alternate (secondary) keys (called CANDIDATE keys in the relational model).
- `Dname VARCHAR(15) UNIQUE;`

→ ensures data integrity by preventing duplicate values in columns that should have unique values, like emails or usernames.

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY,  
    email VARCHAR(255) UNIQUE, --  
    Ensures all emails are unique  
    username VARCHAR(100),  
    UNIQUE(username) -- unique  
    constraint on the 'username' column  
);
```

Specifying Key and Referential Integrity Constraints (cont'd.)

FOREIGN KEY

```
CREATE TABLE Books (  
  book_id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR (255) NOT NULL,  
  author VARCHAR (255) NOT NULL,  
  genre VARCHAR (100),  
  published_year YEAR,  
  is_available BOOLEAN DEFAULT TRUE  
);
```

```
CREATE TABLE Borrowing (  
  loan_id INT NOT NULL AUTO_INCREMENT PRIMARY  
  KEY,  
  book_id INT,  
  member_id INT,  
  borrow_date DATE DEFAULT (CURRENT_DATE),  
  return_date DATE,  
  librarian_id INT,  
  FOREIGN KEY (book_id) REFERENCES Books(book_id),  
  FOREIGN KEY (member_id) REFERENCES  
  Members(member_id),  
  FOREIGN KEY (librarian_id) REFERENCES  
  Librarians(librarian_id)  
);
```

The SELECT-FROM-WHERE

Structure of Basic SQL Queries

■ Basic form of the `SELECT` statement:

```
SELECT    <attribute list>  
FROM      <table list>  
WHERE     <condition>;
```

where

- <attribute list> is a list of attribute names whose values are to be retrieved by the query.
- <table list> is a list of the relation names required to process the query.
- <condition> is a conditional (Boolean) expression that identifies the tuples to be retrieved by the query.

SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 5.7 (Fig. 6.1)

```
CREATE TABLE EMPLOYEE
( Fname          VARCHAR(15)          NOT NULL,
  Minit          CHAR,
  Lname          VARCHAR(15)          NOT NULL,
  Ssn            CHAR(9)              NOT NULL,
  Bdate          DATE,
  Address        VARCHAR(30),
  Sex            CHAR,
  Salary         DECIMAL(10,2),
  Super_ssn      CHAR(9),
  Dno            INT                  NOT NULL,
  PRIMARY KEY (Ssn),
CREATE TABLE DEPARTMENT
( Dname          VARCHAR(15)          NOT NULL,
  Dnumber        INT                  NOT NULL,
  Mgr_ssn        CHAR(9)              NOT NULL,
  Mgr_start_date DATE,
  PRIMARY KEY (Dnumber),
  UNIQUE (Dname),
  FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
( Dnumber        INT                  NOT NULL,
  Dlocation      VARCHAR(15)          NOT NULL,
  PRIMARY KEY (Dnumber, Dlocation),
  FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

continued on ne

SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 5.7 (Fig. 6.1)-continued

```
CREATE TABLE PROJECT
  ( Pname                VARCHAR(15)          NOT NULL,
    Pnumber              INT                  NOT NULL,
    Plocation            VARCHAR(15),
    Dnum                 INT                  NOT NULL,
    PRIMARY KEY (Pnumber),
    UNIQUE (Pname),
    FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );

CREATE TABLE WORKS_ON
  ( Essn                 CHAR(9)              NOT NULL,
    Pno                  INT                  NOT NULL,
    Hours                DECIMAL(3,1)         NOT NULL,
    PRIMARY KEY (Essn, Pno),
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
    FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );

CREATE TABLE DEPENDENT
  ( Essn                 CHAR(9)              NOT NULL,
    Dependent_name        VARCHAR(15)         NOT NULL,
    Sex                   CHAR,
    Bdate                 DATE,
    Relationship            VARCHAR(8),
    PRIMARY KEY (Essn, Dependent_name),
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```


One possible database state for the COMPANY relational database schema (Fig. 5.6)

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

One possible database state for the COMPANY relational database schema – continued (Fig. 5.6)

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

<u>Pname</u>	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Basic Retrieval Queries

<u>Bdate</u>	<u>Address</u>
1965-01-09	731 Fondren, Houston, TX

<u>Fname</u>	<u>Lname</u>	<u>Address</u>
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Query 0. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

Q0: **SELECT** Bdate, Address
 FROM EMPLOYEE
 WHERE Fname='John' **AND** Minit='B' **AND** Lname='Smith';

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

Q1: **SELECT** Fname, Lname, Address
 FROM EMPLOYEE, DEPARTMENT
 WHERE Dname='Research' **AND** Dnumber=Dno;

Query 1 above is an example for select-project-join query with 1 join condition.

select-project-join query: A query that involves only selection and join conditions plus projection attributes is known as a select-project-join query.

Basic Retrieval Queries (Contd.)

(c)

<u>Pnumber</u>	<u>Dnum</u>	<u>Lname</u>	<u>Address</u>	<u>Bdate</u>
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

Q2: **SELECT** Pnumber, Dnum, Lname, Address, Bdate
 FROM PROJECT, DEPARTMENT, EMPLOYEE
 WHERE Dnum=Dnumber **AND** Mgr_ssn=Ssn **AND**
 Plocation='Stafford';

Query 2 is an example for select-project-join query with 2 join conditions.

Here, the join condition $Dnum = Dnumber$ relates a **project** tuple to its controlling **department** tuple, whereas the join condition $Mgr_ssn = Ssn$ relates the controlling **department** tuple to the **employee** tuple who manages that department. Each tuple in the result will be a *combination of one project, one department (that controls the project), and one employee (that manages the department).*

Ambiguous Attribute Names

- Same name can be used for two (or more) attributes in different relations
 - As long as the attributes are in different relations
 - Must **qualify** the attribute name with the relation name to prevent ambiguity

This is done by prefixing the relation name to the attribute name and separating the two by a period.

```
Q1A:  SELECT  Fname, EMPLOYEE.Name, Address
      FROM    EMPLOYEE, DEPARTMENT
      WHERE   DEPARTMENT.Name='Research' AND
              DEPARTMENT.Dnumber=EMPLOYEE.Dnumber;
```

```
Q1A:  SELECT  Fname, EMPLOYEE.Name, Address
      FROM    EMPLOYEE, DEPARTMENT
      WHERE   DEPARTMENT.Name = 'Research' AND
             DEPARTMENT.Dnumber = EMPLOYEE.Dnumber;
```

Fully qualified attribute names can be used for clarity even if there is no ambiguity in attribute names. Above query can be rewritten as:

```
Q1':   SELECT  EMPLOYEE.Fname, EMPLOYEE.LName,
              EMPLOYEE.Address
      FROM    EMPLOYEE, DEPARTMENT
      WHERE   DEPARTMENT.DName = 'Research' AND
             DEPARTMENT.Dnumber = EMPLOYEE.Dno;
```

Aliasing, and Renaming

- **Aliases or tuple variables**

- Declare alternative relation names E and S to refer to the EMPLOYEE relation twice in a query:

1. **Table Aliases in Queries:** We can create a **temporary alias** for a table to refer to a table by a **shorter or alternate name** within the scope of that query.

```
SELECT e.name, e.salary  
FROM employees AS e  
WHERE e.department = 'HR';
```



employees is the original table name, **e** is an alias for employees

Aliasing: we declare alternative relation names **E** and **S**, called **aliases or tuple variables**, for the **EMPLOYEE** relation.

Query 8. For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

Q8: **SELECT** E.Fname, E.Lname, S.Fname, S.Lname
 FROM EMPLOYEE **AS** E, EMPLOYEE **AS** S
 WHERE E.Super_ssn = S.Ssn;

(d)

<u>E.Fname</u>	<u>E.Lname</u>	<u>S.Fname</u>	<u>S.Lname</u>
John	Smith	Franklin	Wong
Franklin	Wong	James	Borg
Alicia	Zelaya	Jennifer	Wallace
Jennifer	Wallace	James	Borg
Ramesh	Narayan	Franklin	Wong
Joyce	English	Franklin	Wong
Ahmad	Jabbar	Jennifer	Wallace

Contd....

```
e.g.: SELECT e1.name AS manager, e2.name AS employee  
FROM employees AS e1  
JOIN employees AS e2 ON e1.id = e2.manager_id  
WHERE e1.department = 'Sales';
```

JOIN is used to combine rows from the **employees** table twice, once as **e1** (for managers) and once as **e2** (for employees).

It is also possible to rename the relation attributes within the query in SQL by giving them aliases.

EMPLOYEE AS E(Fn, Mi, Ln, Ssn, Bd, Addr, Sex, Sal, Sssn, Dno),

Here, Fn becomes an alias for Fname, Mi for Minit, Ln for Lname, and so on.

Queries 9 and 10. Select all EMPLOYEE Ssns (Q9) and all combinations of EMPLOYEE Ssn and DEPARTMENT Dname (Q10) in the database.

Q9: SELECT Ssn
 FROM EMPLOYEE;

Q10: SELECT Ssn, Dname
 FROM EMPLOYEE, DEPARTMENT;

)

SSN
123456789
333445555
999887777
987654321
666884444
453453453
987987987
888665555

Ssn	Dname
123456789	Research
333445555	Research
999887777	Research
987654321	Research
666884444	Research
453453453	Research
987987987	Research
888665555	Research
123456789	Administration
333445555	Administration
999887777	Administration
987654321	Administration
666884444	Administration
453453453	Administration
987987987	Administration
888665555	Administration
123456789	Headquarters
333445555	Headquarters
999887777	Headquarters
987654321	Headquarters
666884444	Headquarters
453453453	Headquarters
987987987	Headquarters
888665555	Headquarters

Use of the Asterisk...

used as a shorthand (or used as a wildcard character in SQL queries), to **select all columns** from a table in a SELECT statement.

`SELECT * FROM employees;` Here, all attributes of the employees table are retrieved.

Q1C: `SELECT *
FROM EMPLOYEE
WHERE Dno = 5;` //retrieves all the attribute values of any EMPLOYEE who works in DEPARTMENT number 5

Q1D: `SELECT *
FROM EMPLOYEE, DEPARTMENT
WHERE Dname = 'Research' AND Dno = Dnumber;` //retrieves all the attributes of an EMPLOYEE and the attributes of the DEPARTMENT in which he or she works for every employee of the 'Research' department

Q10A: `SELECT *
FROM EMPLOYEE, DEPARTMENT;` //Specifies the CROSS PRODUCT of the EMPLOYEE and DEPARTMENT relations.

Result of query Q1C:

(g)

<u>Fname</u>	<u>Minit</u>	<u>Lname</u>	Ssn	<u>Bdate</u>	<u>Address</u>	<u>Sex</u>	<u>Salary</u>	Super_ssn	<u>Dno</u>
John	B	Smith	123456789	1965-09-01	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

Few more queries ...

```
SELECT *  
FROM employees AS e  
JOIN departments AS d ON e.department_id = d.id;
```

```
SELECT * FROM customers WHERE country = 'INDIA';
```

```
SELECT * FROM orders  
JOIN customers ON orders.customer_id = customers.customer_id;
```


Use the keyword **DISTINCT** in the **SELECT** clause

SQL does not automatically eliminate duplicate tuples in query results.

Use the keyword **DISTINCT** in the **SELECT** clause

- Only distinct tuples should remain in the result

Query 11. Retrieve the salary of every employee (Q11) and all distinct salary values (Q11A).

Q11: **SELECT** **ALL** Salary
 FROM **EMPLOYEE;**

Q11A: **SELECT** **DISTINCT** Salary
 FROM **EMPLOYEE;**

(a)

Salary
30000
40000
25000
43000
38000
25000
25000
55000

(b)

Salary
30000
40000
25000
43000
38000
55000

SET operations

UNION: MySQL supports the UNION operator.

UNION combines the results of two or more SELECT queries into a single result set, **removing duplicate rows**.

```
SELECT name FROM employees  
UNION  
SELECT name FROM managers;
```

returns a list of all unique names from both the employees and managers tables.

Query 4. Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
Q4A:  ( SELECT  DISTINCT Pnumber
        FROM      PROJECT, DEPARTMENT, EMPLOYEE
        WHERE      Dnum = Dnumber AND Mgr_ssn = Ssn
                AND      Lname = 'Smith' )

        UNION

        ( SELECT  DISTINCT Pnumber
        FROM      PROJECT, WORKS_ON, EMPLOYEE
        WHERE      Pnumber = Pno AND Essn = Ssn
                AND      Lname = 'Smith' );
```

The first SELECT query retrieves the projects that involve a 'Smith' as manager of the department that controls the project, and the second retrieves the projects that involve a 'Smith' as a worker on the project. Notice that if several employees have the last name 'Smith', the project names involving any of them will be retrieved. Applying the UNION operation to the two SELECT queries gives the desired result.

SQL also has corresponding multiset operations, which are followed by the keyword ALL (UNION ALL, EXCEPT ALL, INTERSECT ALL). Their results are multisets (duplicates are not eliminated). **Try to analyse these operations:**

(a)	R	S	(b)	T	(c)	T
	A	A		A		A
	a1	a1		a1		a2
	a2	a2		a1		a3
	a2	a4		a2		
	a3	a5		a2		
				a2		
				a3	(d)	T
				a4		A
				a5		a1
						a2

Figure 6.5

The results of SQL multiset operations. (a) Two tables, R(A) and S(A).
 (b) R(A) UNION ALL S(A).
 (c) R(A) EXCEPT ALL S(A).
 (d) R(A) INTERSECT ALL S(A).

Note: UNION ALL does not remove duplicates.

Contd....

Note: MySQL does not support the EXCEPT and INTERSECT operators directly, unlike some other database management systems like PostgreSQL or SQL Server.

INTERSECT Equivalent in MySQL: We can use an **INNER JOIN**.

```
SELECT column1                                // This will return the rows that are common in
FROM table1                                   both table1 and table2
INNER JOIN table2 ON table1.column1 = table2.column1;
```

EXCEPT Equivalent in MySQL: We can use **NOT EXISTS** or **NOT IN** constructs

```
SELECT column1
FROM table1
WHERE NOT EXISTS (SELECT 1 FROM table2 WHERE table1.column1 = table2.column1);
```

//This query will return rows from table1 that are not in table2

SET operations

NOT IN: returns the rows from the first query that do not exist in the second query.

```
SELECT name FROM employees  
WHERE name NOT IN (SELECT name FROM managers);
```

This query returns the names of employees who are **not managers**.

SET operations: e.g.2:

employees table:

id	name	department
1	Alice	Sales
2	Bob	HR
3	Charlie	Marketing
4	David	Sales

projects table:

project_id	project_name	employee_id
101	Project A	1
102	Project B	3

```
SELECT e.name AS employee_name, p.project_name FROM employees e
LEFT JOIN projects p ON e.id = p.employee_id;
```

SET operations

LEFT JOIN projects p ON e.id = p.employee_id: This means that for each employee in the employees table, we try to match them with any project in the projects table where e.id = p.employee_id. If an employee is not assigned to a project, the columns from the projects table will be filled with NULL for that row.

Result:

employee_name	project_name
Alice	Project A
Bob	NULL
Charlie	Project B
David	NULL

Substring Pattern Matching and Arithmetic Operators

- **LIKE** comparison operator
 - Used for string **pattern matching**
 - % replaces an arbitrary number of zero or more characters
 - underscore (_) replaces a single character

e.g. 1: `SELECT * FROM employees WHERE name LIKE 'A%';`
//This query finds all employees whose names start with the letter "A".

e.g. 2: `SELECT * FROM employees WHERE name LIKE 'M_x';`
//This query finds all employees whose names are three characters long and start with "M" and end with "x" (e.g., "Max").

e.g. 3: `SELECT * FROM users WHERE email LIKE 'john%@.com';` //This will return emails like "johnsmith@gmail.com", "john123@yahoo.com", etc.

Few more queries....

Query 12. Retrieve all employees whose address is in Houston, Texas.

```
Q12:  SELECT  Fname, Lname
      FROM    EMPLOYEE
      WHERE   Address LIKE '%Houston,TX%';
```

To retrieve all employees who were born during the 1970s, we can use Query Q12A. Here, '7' must be the third character of the string (according to our format for date), so we use the value ' 7 ', with each underscore serving as a placeholder for an arbitrary character.

Query 12A. Find all employees who were born during the 1970s.

```
Q12:  SELECT  Fname, Lname
      FROM    EMPLOYEE
      WHERE   Bdate LIKE '  7      ';
```

Substring Pattern Matching and Arithmetic Operators: BETWEEN comparison operator

- The BETWEEN operator in MySQL is used to filter the result set within a specified range. It is commonly used with numeric, date, or text data types.

e.g. 1: `SELECT * FROM products WHERE price BETWEEN 50 AND 150;`

//Find all products that cost between 50 and 150.

e.g.2: `SELECT * FROM orders WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';`

//Find all orders placed between January 1, 2023, and December 31, 2023.

Substring Pattern Matching and Arithmetic Operators: BETWEEN comparison operator

Query 14. Retrieve all employees in department 5 whose salary is between \$30,000 and \$40,000.

```
Q14:  SELECT  *  
      FROM    EMPLOYEE  
      WHERE   (Salary BETWEEN 30000 AND 40000) AND Dno = 5;
```

The condition (Salary BETWEEN 30000 AND 40000) in Q14 is equivalent to the condition ((Salary >= 30000) AND (Salary <= 40000)).

Arithmetic Operations

- Standard arithmetic operators:
 - Addition (+), subtraction (−), multiplication (*), and division (/) may be included as a part of **SELECT**
- **Query 13.** Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise.

```
SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal  
FROM EMPLOYEE AS E, WORKS_ON AS W, PROJECT AS P  
WHERE E.Ssn=W.Essn AND W.Pno=P.Pnumber AND  
P.Pname='ProductX';
```

Note: For string data types, the concatenate operator || can be used in a query to append two string values.

Arithmetic Operations

e.g. 2: `SELECT revenue / 12 AS monthly_revenue FROM sales;`

e.g. 3: `SELECT (revenue - cost) / (employees * 12) AS profit_per_employee FROM company_data;`

Ordering of Query Results

SQL allows the user to **order the tuples** in the result of a query by the values of one or more of the attributes that appear in the query result, by using the **ORDER BY** clause.

- Use **ORDER BY** clause
 - Keyword **DESC** to see result in a descending order of values
 - Keyword **ASC** to specify ascending order explicitly
 - Typically placed at the end of the query

```
ORDER BY D.Dname DESC, E.Lname ASC,  
         E.Fname ASC
```

Basic SQL Retrieval Query Block

```
SELECT    <attribute list>  
FROM      <table list>  
[ WHERE   <condition> ]  
[ ORDER BY <attribute list> ];
```

Query 15. Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, then first name.

```
Q15:  SELECT    D.Dname, E.Lname, E.Fname, P.Pname
      FROM      DEPARTMENT AS D, EMPLOYEE AS E, WORKS_ON AS W,
              PROJECT AS P
      WHERE     D.Dnumber = E.Dno AND E.Ssn = W.Essn AND W.Pno =
              P.Pnumber
      ORDER BY  D.Dname, E.Lname, E.Fname;
```

The default order is in **ascending order of values**. We can specify the keyword **DESC** if we want to see the result in a descending order of values. The keyword **ASC** can be used to specify ascending order explicitly. For example:

```
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC
```

INSERT, DELETE, and UPDATE Statements in SQL

- Three commands used to modify the database:
 - INSERT, DELETE, and UPDATE
- INSERT typically inserts a tuple (row) in a relation (table)
- UPDATE may update a number of tuples (rows) in a relation (table) that satisfy the condition
- DELETE may also update a number of tuples (rows) in a relation (table) that satisfy the condition

INSERT

- In its simplest form, it is used to add one or more tuples to a relation
- Attribute values should be listed in the same order as the attributes were specified in the **CREATE TABLE** command
- Constraints on data types are observed automatically
- Any integrity constraints as a part of the DDL specification are enforced

The INSERT Command

- Specify the relation name and a list of values for the tuple. All values including nulls are supplied.

```
U1:  INSERT INTO  EMPLOYEE  
      VALUES      ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98  
                    Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );
```

- The variation below inserts multiple tuples where a new table is loaded values from the result of a query.

```
U3B:  INSERT INTO  WORKS_ON_INFO ( Emp_name, Proj_name,  
                                     Hours_per_week )  
      SELECT      E.Lname, P.Pname, W.Hours  
      FROM        PROJECT P, WORKS_ON W, EMPLOYEE E  
      WHERE       P.Pnumber=W.Pno AND W.Essn=E.Ssn;
```

In MySQL, if an attribute (or column) is not specified in an INSERT command, the value assigned to that attribute depends on its data type and whether it has a default value.

Default value: If a default is set, that value will be used.

NULL allowed: If the column allows NULLs, it will be set to NULL.

Not NULL, no default: The INSERT will fail if the column is required but not specified.

AUTO_INCREMENT: Automatically assigns the next incremented value.

A DBMS that fully implements SQL should support and enforce all the integrity constraints that can be specified in the DDL. For example, if we issue the command in U2 on the database shown in Figure 5.6, the DBMS should *reject* the operation because no DEPARTMENT tuple exists in the database with Dnumber = 2. Similarly, U2A would be *rejected* because no Ssn value is provided and it is the primary key, which cannot be NULL.

U2: **INSERT INTO** EMPLOYEE (Fname, Lname, Ssn, Dno)
 VALUES ('Robert', 'Hatcher', '980760540', 2);
 (U2 is rejected if referential integrity checking is provided by DBMS.)

U2A: **INSERT INTO** EMPLOYEE (Fname, Lname, Dno)
 VALUES ('Robert', 'Hatcher', 5);
 (U2A is rejected if NOT NULL checking is provided by DBMS.)

DELETE

- Removes tuples from a relation
 - Includes a WHERE-clause to select the tuples to be deleted
 - Referential integrity should be enforced
 - Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint)
 - A missing WHERE-clause specifies that *all tuples* in the relation are to be deleted; the table then becomes an empty table
 - The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause

The DELETE Command

- Removes tuples from a relation
 - Includes a **WHERE** clause to select the tuples to be deleted. The number of tuples deleted will vary.

U4A:	DELETE FROM	EMPLOYEE
	WHERE	Lname='Brown';
U4B:	DELETE FROM	EMPLOYEE
	WHERE	Ssn='123456789';
U4C:	DELETE FROM	EMPLOYEE
	WHERE	Dno=5;
U4D:	DELETE FROM	EMPLOYEE;

The DELETE commands in U4A to U4D, if applied independently to the database states shown in Figure 5.6, will delete zero, one, four, and all tuples, respectively, from the EMPLOYEE relation:

The DELETE Command

When we try to delete a record in a table that is referenced by another table (via a foreign key) (i.e. when we have **referential integrity** established between tables using **foreign key constraints**), MySQL will not allow the deletion by default if there is still a dependency (i.e., a record in a child table that references the record in the parent table). This can be handled using specific **foreign key actions**.

1. **CASCADE**: When the referenced record in the parent table is deleted, all matching records in the child table are also deleted or updated to maintain referential integrity.
 - i. **ON DELETE CASCADE**: Automatically deletes related records in the child table when a record in the parent table is deleted.
 - ii. **ON UPDATE CASCADE**: Automatically updates related records in the child table when a record in the parent table is updated.

The DELETE Command

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  order_date DATE  
);
```

```
CREATE TABLE order_items (  
  item_id INT PRIMARY KEY,  
  order_id INT,  
  product_name VARCHAR(100),  
  quantity INT,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id)  
  ON DELETE CASCADE  
);
```

The **ON DELETE CASCADE** clause ensures that if a record in orders is deleted, the related records in order_items are also deleted automatically.

DELETE FROM orders WHERE order_id = 1;

1. Before deletion:

- orders table:

order_id	customer_name	order_date
1	John Doe	2025-02-01
2	Jane Smith	2025-02-02

- order_items table:

item_id	order_id	product_name	quantity
101	1	Laptop	1
102	1	Mouse	2
103	2	Keyboard	1

2. After running DELETE FROM orders WHERE order_id = 1 ;:

- orders table:

order_id	customer_name	order_date
2	Jane Smith	2025-02-02

- order_items table:

item_id	order_id	product_name	quantity
103	2	Keyboard	1

The DELETE Command

ON UPDATE CASCADE

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  order_date DATE  
);
```

```
CREATE TABLE order_items (  
  item_id INT PRIMARY KEY,  
  order_id INT,  
  product_name VARCHAR(100),  
  quantity INT,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id)  
  ON UPDATE CASCADE  
);
```

UPDATE CASCADE ensures that if the primary key in the parent table (orders table in this example) is updated, the foreign key in the child table (order_items table) is also updated automatically.

UPDATE orders SET order_id = 10 WHERE order_id = 1;

1. Before update:

- orders table:

order_id	customer_name	order_date
1	John Doe	2025-02-01
2	Jane Smith	2025-02-02

- order_items table:

item_id	order_id	product_name	quantity
101	1	Laptop	1
102	1	Mouse	2
103	2	Keyboard	1

2. After running UPDATE orders SET order_id = 10 WHERE order_id = 1; :

- orders table:

order_id	customer_name	order_date
10	John Doe	2025-02-01
2	Jane Smith	2025-02-02

- order_items table:

item_id	order_id	product_name	quantity
101	10	Laptop	1
102	10	Mouse	2
103	2	↓ board	1

The DELETE Command

Combining Both **ON DELETE CASCADE** and **ON UPDATE CASCADE**: we can also use both **ON DELETE CASCADE** and **ON UPDATE CASCADE** together in your foreign key constraints to ensure that both deletions and updates in the parent table automatically cascade to the child table.

UPDATE

- Used to modify attribute values of one or more selected tuples
- A WHERE-clause selects the tuples to be modified
- An additional SET-clause specifies the attributes to be modified and their new values
- Each command modifies tuples *in the same relation*
- Referential integrity specified as part of DDL specification is enforced

UPDATE (contd.)

- Example: Change the location and controlling department number of project number 10 to 'Bellaire' and 5, respectively

```
U5:      UPDATE    PROJECT  
         SET       PLOCATION = 'Bellaire',  
                DNUM = 5  
         WHERE     PNUMBER=10
```

UPDATE (contd.)

- Example: Give all employees in the 'Research' department a 10% raise in salary.

```
U6:      UPDATE      EMPLOYEE
         SET          Salary = Salary * 1.1
         WHERE        Dno = 5;
```

- In this request, the modified SALARY value depends on the original SALARY value in each tuple
 - The reference to the SALARY attribute on the right of = refers to the old SALARY value before modification
 - The reference to the SALARY attribute on the left of = refers to the new SALARY value after modification

THANK YOU